

03-Java并发

Java 并发（JUC）面试题

1. 线程基础

问：创建线程的方式？

答：1. 继承 Thread ，重写 run() 2. 实现 Runnable （推荐，解耦任务与线程） 3. 实现 Callable + FutureTask （可有返回值） 4. 线程池 （生产环境推荐）

问：线程有哪些状态？

答：

NEW → RUNNABLE → BLOCKED / WAITING / TIMED_WAITING → TERMINATED

状态	说明
NEW	已创建未 start
RUNNABLE	可运行（含 Running 和 Ready）
BLOCKED	等待 synchronized 锁
WAITING	无限等待（wait/join/park）
TIMED_WAITING	超时等待（sleep/wait(time)/join(time)）
TERMINATED	已终止

问：sleep 和 wait 的区别？

答：

对比	sleep	wait
所属类	Thread	Object
释放锁	不释放	释放
唤醒	超时自动	notify/notifyAll
使用场景	暂停执行	线程通信

问：start 和 run 的区别？

答：– start()：启动新线程，JVM 调用 run()，只能调用一次 – run()：普通方法调用，在当前线程执行，可多次调用

问：Java 线程和操作系统线程一样吗？

答：分两种情况：

类型	映射关系	特点
平台线程 (Platform Thread)	1:1 映射 OS 线程	JDK 传统模型，栈约 1MB， 创建/切换开销大
虚拟线程 (Virtual Thread, JDK 21+)	M:N 映射到少量载体线程	轻量 (几 KB)，适合 IO 密集 型，可创建百万级

结论：JDK 21 之前「Java 线程 = OS 线程」成立；使用虚拟线程后，由 JVM 调度，不再一一对应。

问：如何正确停止一个线程？

答：不推荐：Thread.stop() 已废弃，可能破坏锁状态和数据一致性。

推荐：协作式中断 —— thread.interrupt() + 线程内检查中断状态。

```
// 方式1: 捕获 InterruptedException (sleep/wait/join 等)
try {
    Thread.sleep(1000);
} catch (InterruptedException e) {
    Thread.currentThread().interrupt(); // 恢复中断标志
    return;
}

// 方式2: 轮询中断标志
while (!Thread.currentThread().isInterrupted()) {
    // 执行业务
}
```

注意：用 Thread.currentThread().isInterrupted() 而非 Thread.interrupted()（后者会清除中断标志）。

问：BLOCKED 和 WAITING 有什么区别？

答：

对比	BLOCKED	WAITING
触发	竞争 synchronized 锁失败	主动调用 wait/join/park
唤醒	锁释放后自动竞争	需 notify/notifyAll/unpark 显式唤醒
本质	被动等锁	主动等条件

问：notify 和 notifyAll 的区别？

答：– notify()：唤醒一个等待线程（HotSpot 实际近似 FIFO），其余仍在 WAITING – notifyAll()：唤醒所有等待线程，它们竞争锁，只有一个获得

风险：只用 notify 时，若被唤醒线程条件不满足又 wait，且没有其他 notify，可能永久等待（需配合 while 循环检查条件）。

```
synchronized (lock) {  
    while (!condition) { // 必须用 while, 不用 if  
        lock.wait();  
    }  
}
```

问：线程间通信方式有哪些？

答：

方式	说明	场景
共享变量 + volatile	保证可见性	简单标志位
wait/notify	Object 监视器机制	生产者-消费者
Lock + Condition	多个等待队列，更灵活	复杂条件等待
BlockingQueue	线程安全阻塞队列	生产者-消费者（推荐）
CountDownLatch/CyclicBarrier	JUC 工具类	协调多线程阶段

问：sleep 会释放 CPU 吗？会释放锁吗？

答：– 释放 CPU：是，线程进入 TIMED_WAITING，让出时间片 – 释放锁：否，sleep 期间仍持有 synchronized 锁

2. synchronized

问：synchronized 的作用和原理？

答：保证 原子性、可见性、有序性（通过 Monitor 监视器锁）。

锁对象：– 实例方法 → 锁 this – 静态方法 → 锁 Class 对象 – 代码块 → 锁指定对象

锁升级（JDK 6+ 优化）：

无锁 → 偏向锁 → 轻量级锁 → 重量级锁

- 偏向锁：同一线程多次获取，CAS 记录线程 ID

- 轻量级锁：少量竞争，CAS 自旋
- 重量级锁：激烈竞争，挂起线程，依赖 OS Mutex

问：synchronized 锁静态方法和普通方法有什么区别？

答：

类型	锁对象	说明
实例方法	this （当前实例）	不同实例互不干扰
静态方法	Class 对象	全 JVM 唯一，所有实例共享
代码块	指定对象	可细粒度控制

注意：静态 synchronized 与实例 synchronized 不互斥（锁对象不同）。

问：synchronized 支持可重入吗？如何实现？

答：支持。同一线程可多次获取同一把锁。

实现：Monitor 对象记录 持有锁的线程 和 重入次数 (recursions)： – 首次获取：记录 owner，recursions = 1 – 同线程再次进入：recursions++ – 退出：recursions–，到 0 时释放锁

问：除了 synchronized，还有哪些线程同步方式？

答：

方式	说明
ReentrantLock	显式锁，可中断、可公平、多 Condition
ReadWriteLock	读多写少，读写分离
Semaphore	信号量，控制并发数
CountDownLatch / CyclicBarrier	协调多线程
Atomic 类	CAS 无锁原子操作
volatile	可见性 + 禁止重排（不保证原子性）
BlockingQueue	阻塞队列实现生产者-消费者

3. volatile

问：volatile 的作用？能保证原子性吗？

答：作用： 1. 可见性：修改立即刷新到主内存，读取从主内存读 2. 禁止指令重排：通过内存屏障实现
不能保证原子性： i++ 是读-改-写三步，需 synchronized 或 AtomicInteger。

原理：MESI 缓存一致性协议 + 内存屏障（LoadLoad、StoreStore、LoadStore、StoreLoad）。

使用场景：状态标志位、双重检查锁定（DCL）单例。

问：volatile 和 synchronized 的区别？

答：

对比	volatile	synchronized
作用	可见性 + 禁止重排	原子性 + 可见性 + 有序性
粒度	变量级	代码块/方法级
阻塞	不阻塞	未获锁线程阻塞
性能	轻量	较重（涉及锁竞争）
复合操作	不能保证 i++ 原子性	可以

选型：单一状态标志用 volatile；复合操作或临界区用 synchronized/Lock。

4. CAS 与 AQS

问：什么是 CAS？有什么问题？

答：CAS（Compare And Swap）：比较内存值与期望值，相等则更新，CPU 原子指令。

问题：1. **ABA 问题：**值 A→B→A，CAS 认为未变。解决：AtomicStampedReference 加版本号 2. **自旋开销：**长时间 CAS 失败消耗 CPU 3. **只能保证一个变量原子性**

问：悲观锁和乐观锁的区别？Java 如何实现乐观锁？

答：

对比	悲观锁	乐观锁
假设	竞争总会发生	竞争不常发生
策略	先加锁再操作	先操作，提交时 CAS 校验
代表	synchronized、ReentrantLock	Atomic 类、CAS、版本号

Java 乐观锁实现：1. **CAS：**AtomicInteger、AtomicReference 等 2. **版本号：**数据库 version 字段、MyBatis-Plus @Version 3. **AtomicStampedReference：**CAS + 版本号，解决 ABA

为什么不能全用 CAS：高竞争下大量自旋浪费 CPU，需升级为阻塞锁。

问：CAS 和 AQS 有什么关系？

答：– CAS：底层原子操作，AQS 用 CAS 修改 state 和队列指针 – AQS：基于 CAS + CLH 队列的同步框架，封装锁的获取/释放流程 – 关系：CAS 是原子手段，AQS 是同步框架，ReentrantLock 等基于 AQS 实现

问：AQS 是什么？

答：AbstractQueuedSynchronizer，JUC 核心框架，基于 volatile state + CLH 双向队列。

- state：同步状态（如 ReentrantLock 的重入次数）
- 队列：封装获取锁失败的线程，FIFO 排队

子类：ReentrantLock、Semaphore、CountDownLatch、ReentrantReadWriteLock。

问：ReentrantLock 和 synchronized 的区别？

答：

对比	synchronized	ReentrantLock
实现	JVM 内置	AQS
释放锁	自动	手动 unlock
可中断	否	lockInterruptibly
公平锁	非公平	可选公平/非公平
多条件	单一 wait/notify	多个 Condition
性能	JDK 6 后差距小	略灵活

5. 线程池

问：线程池 7 大参数和执行流程？

答：

```
new ThreadPoolExecutor(  
    corePoolSize,      // 核心线程数  
    maximumPoolSize,   // 最大线程数  
    keepAliveTime,     // 非核心线程空闲存活时间  
    unit,              // 时间单位  
    workQueue,         // 任务队列  
    threadFactory,     // 线程工厂  
    handler,           // 拒绝策略  
);
```

执行流程： 1. 线程数 < corePoolSize → 创建核心线程 2. 核心线程满 → 任务入队列 3. 队列满 → 创建非核心线程（至 maximumPoolSize） 4. 线程数达最大且队列满 → 执行拒绝策略

拒绝策略： – AbortPolicy：抛异常（默认） – CallerRunsPolicy：调用者线程执行 – DiscardPolicy：丢弃 – DiscardOldestPolicy：丢弃队列最旧任务

问：为什么不推荐 Executors 创建线程池？

答： – newFixedThreadPool / newSingleThreadExecutor：队列 LinkedBlockingQueue 无界 → OOM
– newCachedThreadPool：maximumPoolSize = Integer.MAX_VALUE → 线程数无限 → OOM

生产环境用 ThreadPoolExecutor 显式指定参数。

问：常见线程池类型有哪些？

答：

类型	特点	风险
FixedThreadPool	固定线程数，无界 LinkedBlockingQueue	队列堆积 OOM
CachedThreadPool	线程数无上限，60s 回收， SynchronousQueue	大量任务时线程爆炸 OOM
SingleThreadExecutor	单线程，任务按提交顺序执行	无界队列 OOM
ScheduledThreadPool	定时/周期性任务， DelayedWorkQueue	需控制任务数

阿里规范：禁止直接使用 Executors，手动 new ThreadPoolExecutor 并指定有界队列。

问：shutdown 和 shutdownNow 的区别？

答：

方法	状态	行为
shutdown()	SHUTDOWN	不再接受新任务；已提交队列任务继续执行；中断空闲线程
shutdownNow()	STOP	尝试中断所有线程；丢弃队列中未执行任务；返回未执行列表

优雅关闭推荐：

```
executor.shutdown();
if (!executor.awaitTermination(60, TimeUnit.SECONDS)) {
    executor.shutdownNow();
}
```

问：线程池任务可以取消吗？submit 和 execute 区别？

答：submit() 返回 Future，可通过 future.cancel(true) 取消（mayInterruptIfRunning=true 会中断执行中线程）。

对比	execute	submit
返回值	void	Future
异常	未捕获则线程终止，打印栈	封装在 Future 中，需 get() 才抛出
取消	不支持	支持 Future.cancel

问：corePoolSize 可以设为 0 吗？

答：可以。任务到来时先入队，若当前无线程则创建非核心线程执行（即使 core=0）。CachedThreadPool 就是 core=0、max=Integer.MAX_VALUE 的典型。

问：线程池用了哪些设计模式？

答：

模式	体现
工厂模式	Executors.newFixedThreadPool() 封装创建
享元模式	复用核心线程执行多个任务
生产者-消费者	提交任务（生产者）→ 队列 → 工作线程（消费者）
模板方法	execute() 固定流程，钩子 beforeExecute / afterExecute 可扩展

6. 并发工具类

问：CountDownLatch、CyclicBarrier、Semaphore 区别？

答：

工具	作用	可重用	典型场景
CountDownLatch	一个/多个线程等 N 个任务完成	否	主线程等子任务
CyclicBarrier	N 个线程互相等到齐再继续	是	多线程分阶段计算
Semaphore	控制同时访问资源的线程数	是	限流、连接池

问：ThreadLocal 原理和内存泄漏？

答：每个 Thread 持有 ThreadLocalMap，key 是 ThreadLocal（弱引用），value 是线程本地值。

内存泄漏：key 被 GC 后，value 仍被强引用无法回收。解决：使用完调用 `threadLocal.remove()`。

底层结构：

```
Thread
├── ThreadLocalMap (key: ThreadLocal 弱引用, value: 强引用)
│   └── Entry[] 数组, 开放寻址法解决冲突
```

InheritableThreadLocal：子线程可继承父线程的值（线程池场景仍可能有问题，慎用）。

最佳实践：线程池环境下必须在 `finally` 中 `remove()`，否则 value 随线程复用永久泄漏。

问：怎么保证多线程安全？

答：

层面	手段
互斥	synchronized、Lock
无锁	Atomic 类、CAS
可见性	volatile、synchronized、final
线程安全容器	ConcurrentHashMap、CopyOnWriteArrayList、BlockingQueue
协调工具	CountDownLatch、Semaphore、CyclicBarrier
设计	不可变对象、ThreadLocal 隔离、减少共享

7. 死锁

问：死锁的四个条件？如何避免？

答：四个必要条件（同时满足才死锁）： 1. 互斥 2. 占有且等待 3. 不可抢占 4. 循环等待

避免： - 固定加锁顺序 - 超时获取锁（tryLock） - 银行家算法 - 死锁检测

排查： `jstack <pid>` 查看 `Found one Java-level deadlock`。

8. 原子类

问：AtomicInteger 如何保证原子性？

答：基于 CAS + volatile： - volatile 保证 value 可见性 - `Unsafe.compareAndSwapInt()` 保证更新原子性

常见： `AtomicInteger`、`AtomicLong`、`AtomicReference`、`LongAdder`（高并发计数更优）。

9. JMM 详解

问：什么是 Java 内存模型（JMM）？它解决什么问题？

答：JMM 定义了 **线程与主内存交互的规则**，屏蔽硬件差异，保证并发程序的 **可见性、有序性、原子性**。

主内存 vs 工作内存：– 所有变量存于主内存 – 每个线程有自己的工作内存（CPU 缓存），存变量副本 – 线程对变量的操作在工作内存进行，需同步回主内存

三大特性：

特性	说明	保证方式
原子性	操作不可分割	synchronized、Lock、Atomic
可见性	修改对其他线程立即可见	volatile、synchronized、final
有序性	禁止指令重排	volatile、happens-before

问：happens-before 原则有哪些？

答：JMM 定义的偏序关系，保证可见性：1. **程序顺序规则**：同一线程中，前面的操作 happens-before 后面的 2. **锁规则**：unlock happens-before 后续 lock 3. **volatile 规则**：写 volatile happens-before 后续读 4. **传递性**：A hb B, B hb C → A hb C 5. **线程 start**：Thread.start() happens-before 线程内任意操作 6. **线程 join**：线程内操作 happens-before Thread.join() 返回 7. **中断规则**：interrupt() happens-before 检测到中断 8. **对象终结**：构造器完成 happens-before finalize()

问：指令重排是什么？as-if-serial 和 happens-before 的关系？

答：编译器和 CPU 为优化性能可能 **重排指令**，不影响单线程结果（as-if-serial 语义），但多线程下可能出问题。

例子：DCL 单例中，new Singleton() 分三步（分配内存、初始化、引用赋值），可能重排为 1→3→2，其他线程看到未初始化的对象。

解决：volatile 禁止重排（插入内存屏障）。

10. synchronized 锁升级完整过程

问：synchronized 锁升级完整过程是怎样的？

答：对象头 Mark Word 存储锁信息，升级不可逆（只能升级不能降级）：

无锁 → 偏向锁 → 轻量级锁 → 重量级锁

1. **偏向锁**：– 第一个线程获取锁，CAS 将 Mark Word 中的线程 ID 设为自己 – 同一线程再次进入，无需 CAS，直接执行 – 其他线程竞争 → 撤销偏向锁，升级为轻量级锁

2. 轻量级锁：– 竞争不激烈，线程 CAS 将 Mark Word 复制到栈帧 Lock Record – 成功则获取锁；失败则自旋等待 – 自旋超过阈值或竞争激烈 → 膨胀为重量级锁
3. 重量级锁：– Mark Word 指向 Monitor 对象 – 未获取锁的线程进入 EntryList 阻塞，依赖 OS Mutex，用户态↔内核态切换开销大

问：JDK 15 为什么废弃偏向锁？现在默认行为是什么？

答：废弃原因（JEP 374）：1. 偏向锁撤销（revocation）成本高，Safepoint 暂停影响延迟 2. 现代应用多线程竞争激烈，偏向锁收益低 3. 与某些 GC（如 ZGC）配合复杂

JDK 15+ 默认：偏向锁 默认禁用（`-XX:-UseBiasedLocking`），锁升级路径变为：

无锁 → 轻量级锁 → 重量级锁

追问：可通过 `-XX:+UseBiasedLocking` 手动开启，但后续版本可能完全移除。

11. AQS 详解

问：AQS 核心数据结构和工作流程？

答：核心字段：– `volatile int state`：同步状态 – CLH 双向队列：封装等待线程的 Node 链表

Node 状态：WAITING、CANCELLED、CONDITION 等。

获取锁流程（独占模式）：1. `tryAcquire()`：CAS 修改 state（子类实现） 2. 失败 → 封装为 Node 入 CLH 队列尾部 3. 自旋检查前驱是否为 head 且 `tryAcquire` 成功 4. 失败 → `LockSupport.park()` 阻塞 5. 释放锁 → `tryRelease()` → `unpark()` 唤醒后继节点

共享模式（Semaphore、CountDownLatch）：`tryAcquireShared` / `tryReleaseShared`。

问：ReentrantLock 如何实现可重入？

答：AQS 的 `state` 记录重入次数：– 首次获取：state 0→1，记录 owner 线程 – 同线程再次获取：state++ – 释放：state–，到 0 时真正释放锁，唤醒等待线程

12. DCL 单例

问：双重检查锁定（DCL）单例为什么需要 `volatile`？

答：

```
private volatile static Singleton instance;

public static Singleton getInstance() {
    if (instance == null) {
        synchronized (Singleton.class) {
            if (instance == null) {
                instance = new Singleton(); // 非原子操作
            }
        }
    }
    return instance;
}
```

`new Singleton()` 三步：分配内存 → 初始化 → 引用赋值。可能重排为 1→3→2，其他线程看到未初始化的对象。

volatile 禁止指令重排，保证初始化完成后才赋值引用。

问：DCL 还有其他实现方式吗？

答：1. 静态内部类（推荐）：JVM 类加载机制保证线程安全，延迟加载 2. 枚举单例：Effective Java 推荐，防反射和序列化 3. Holder 模式：`private static class Holder { static final Singleton INSTANCE = new Singleton(); }`

13. 公平锁与非公平锁

问：公平锁和非公平锁的区别？`ReentrantLock` 默认哪种？

答：

对比	公平锁	非公平锁
获取顺序	FIFO，先来先得	允许插队
吞吐量	低（频繁上下文切换）	高
饥饿	不会	可能（概率低）
默认	<code>ReentrantLock(false)</code>	非公平

非公平锁优势：新线程可直接 CAS 抢锁，减少唤醒开销。大多数场景用非公平锁。

synchronized：只有非公平锁。

ReentrantLock 公平锁实现：公平锁在 `tryAcquire` 时多判断 `hasQueuedPredecessors()` ——队列中已有等待线程则不再 CAS 抢锁；非公平锁直接 CAS 尝试获取。

非公平锁吞吐量更高：减少线程休眠/唤醒的内核态切换；新线程可直接 CAS 抢锁，避免队首唤醒开销。

14. 线程池参数经验值

问：线程池参数如何设置？有哪些经验值？

答：CPU 密集型：`corePoolSize = CPU 核数 + 1`（+1 防止缺页中断等暂停）

IO 密集型：`corePoolSize = CPU 核数 × 2` 或 `CPU 核数 / (1 - 阻塞系数)`，阻塞系数 0.8~0.9

队列选择：

队列	特点	场景
<code>ArrayBlockingQueue</code>	有界，数组	通用，防 OOM
<code>LinkedBlockingQueue</code>	可有界/无界	注意设容量
<code>SynchronousQueue</code>	不存储，直接交付	Cached 风格，高吞吐
<code>DelayedWorkQueue</code>	延迟	<code>ScheduledThreadPool</code>

拒绝策略：生产推荐 `CallerRunsPolicy`（降级到调用者执行，不丢任务）或自定义（打日志 + 告警）。

示例（8 核，Web 应用）：

```
corePoolSize = 16, maximumPoolSize = 32,
queue = new ArrayBlockingQueue<>(500),
keepAliveTime = 60s, handler = new CallerRunsPolicy()
```

问：线程池核心线程会被回收吗？

答：默认不会。`allowCoreThreadTimeOut = true` 时，核心线程空闲超过 `keepAliveTime` 也会被回收。

15. CompletableFuture

问：CompletableFuture 是什么？和 Future 有什么区别？

答：JDK 8 异步编程工具，实现 `Future` + `CompletionStage`。

对比	Future	CompletableFuture
阻塞获取	<code>get()</code> 阻塞	回调 non-blocking
链式组合	不支持	<code>thenApply/thenCompose</code>
多任务组合	不支持	<code>allOf/anyOf</code>
异常处理	需 try-catch	<code>exceptionally/handle</code>

问：CompletableFuture 常用 API 和场景？

答：

```
// 异步执行
CompletableFuture.supplyAsync(() -> queryDB(), executor);

// 链式处理
future.thenApply(result -> transform(result))
      .thenAccept(result -> save(result));

// 组合
CompletableFuture.allOf(f1, f2, f3).join();
CompletableFuture.anyOf(f1, f2); // 任一完成

// 异常
future.exceptionally(ex -> defaultValue);
```

场景：并行调用多个 RPC/DB、异步编排、超时控制（`orTimeout` JDK 9+）。

注意：默认用 `ForkJoinPool.commonPool()`，生产环境应指定自定义线程池。

16. 其他深入

问：LongAdder 和 AtomicLong 的区别？

答：高并发下 LongAdder 性能更好： – AtomicLong：单变量 CAS，竞争激烈时自旋多 – LongAdder：分段累加（`base + Cell[]`），最后 `sum()` 合并

适用：高并发计数（QPS 统计）；需要精确 CAS 语义用 AtomicLong。

问：StampedLock 了解吗？

答：JDK 8 读写锁优化： – 乐观读：无锁，读完后验证 stamp，无写则成功 – 悲观读/写：类似 ReentrantReadWriteLock

适合 读多写少 且允许短暂不一致的场景。不可重入，不可用于锁升级。

问：ReentrantReadWriteLock 是什么？适用场景？

答：读写锁，允许多个读线程同时访问，写线程独占。

锁	兼容性
读 + 读	允许
读 + 写	互斥
写 + 写	互斥

适用：读远多于写（缓存、配置中心）。注意：写锁饥饿问题，高写场景不如 StampedLock 或 synchronized。

锁降级：持有写锁 → 获取读锁 → 释放写锁（支持）；锁升级（读→写）不支持，会死锁。

17. 并发场景题

问：3 个线程并发执行，1 个线程等它们全部完成再继续，怎么实现？

答：用 `CountDownLatch`：

```
CountDownLatch latch = new CountDownLatch(3);
for (int i = 0; i < 3; i++) {
    new Thread(() -> {
        try { doWork(); } finally { latch.countDown(); }
    }).start();
}
latch.await(); // 主线程阻塞，计数到 0 后继续
System.out.println("全部完成");
```

也可用 `CompletableFuture.allOf(f1, f2, f3).join()` 或 `CyclicBarrier(3)`。

问：两个线程对 `int` 变量各加 50 次，结果可能是什么？

答：理论正确值 100，但无同步时 可能 < 100。

原因：`i++` 是读-改-写三步，非原子操作，并发下会丢失更新。

解决：`synchronized`、`AtomicInteger`、`Lock`。

问：多线程交替打印奇偶数怎么实现？

答：用 `synchronized + wait/notify`，配合 `while` 循环检查条件：

```
synchronized (lock) {
    while (count <= MAX) {
        if (count % 2 == 目标) {
            System.out.println(count++);
            lock.notify();
        } else {
            lock.wait();
        }
    }
}
```

也可用 `ReentrantLock + Condition`（两个 `Condition` 分别等待奇/偶）、`Semaphore` 或 `BlockingQueue`。

18. JUC 常用类速查

问：juc 包下常用的类有哪些？

答：

分类	类
锁	ReentrantLock、ReentrantReadWriteLock、StampedLock
原子	AtomicInteger、AtomicReference、LongAdder
工具	CountDownLatch、CyclicBarrier、Semaphore
容器	ConcurrentHashMap、CopyOnWriteArrayList、BlockingQueue
线程池	ThreadPoolExecutor、ForkJoinPool
异步	CompletableFuture、FutureTask
ThreadLocal	ThreadLocal、InheritableThreadLocal

大厂追问

1. volatile 能禁止哪些重排？

- 1. 写 volatile 前的操作不会重排到写之后
- 2. 读 volatile 后的操作不会重排到读之前
- 3. 写 volatile 前的操作不会重排到读 volatile 之后

2. AQS 为什么用 CLH 队列变体？

CLH 队列减少锁竞争，每个节点自旋等前驱释放。AQS 改为阻塞（park）而非自旋，节省 CPU。

3. 线程池 submit 和 execute 区别？

execute 直接执行 Runnable；submit 返回 Future，异常被封装在 Future 中，需 get() 才能抛出，否则被吞掉。

4. ThreadLocal 为什么 key 是弱引用？

防止 ThreadLocal 对象无法回收。但 value 是强引用，若 Thread 长期存活（如线程池），value 仍会泄漏。用完必须 remove()。

5. synchronized 锁优化后还有性能问题吗？

JDK 15+ 偏向锁默认关闭，轻量级锁自旋过多仍升级重量级锁。高竞争场景 ReentrantLock 可 tryLock 超时、公平锁等更灵活。

6. 公平锁 tryLock 是公平的吗？

不是。tryLock() 调用 nonfairTryAcquire()，允许插队，与构造器设置的公平/非公平无关。

7. CountdownLatch 和 CyclicBarrier 选型？

- 一个等多个完成 → CountdownLatch（不可重用）
- 多个互相等齐再进入下一阶段 → CyclicBarrier（可重用，支持 barrierAction）

8. ForkJoinPool 和 ThreadPoolExecutor 区别？

ForkJoinPool 基于[工作窃取](#)（Work-Stealing），适合[递归分治任务](#)（如大数组排序）；ThreadPoolExecutor 适合通用任务队列模型。

9. 虚拟线程下 synchronized 有什么问题？

虚拟线程在 synchronized 块内阻塞 IO 时会 [pin 到载体线程](#)，无法释放 OS 线程，失去虚拟线程优势。IO 阻塞场景推荐 ReentrantLock。

10. 如何排查线上死锁？

1. jstack <pid> 搜索 Found one Java-level deadlock
2. Arthas: thread -b 找阻塞线程
3. 预防：固定加锁顺序、tryLock 超时、避免嵌套锁

11. LongAdder 什么时候不能用？

需要 [CAS 精确语义](#)（如 compareAndSet 判断特定值）时用 AtomicLong；LongAdder 的 sum() 是近似值，高并发下不同线程 Cell 合并有短暂不一致。